



FIXFLIP

شرح Authentication Flow

Register - Login - JWT Authorization

ASP.NET Identity + UserManager + SignInManager + JWT

هدف الدليل: شرح إنشاء الحساب، والتحقق من المستخدم عند تسجيل الدخول، ثم استخدام JWT Token للوصول إلى Endpoints المحمية.

ملخص الرحلة

REGISTER

Create User -> ASP.NET Identity -> Store in SQL Database

LOGIN

Find User -> Check Password -> Generate JWT Token

AFTER LOGIN

Bearer Token -> [Authorize] -> Protected Endpoint

إعدادات لعرض ومناقشة مشروع FixFlip

أولاً: Register Flow

CREATE A NEW USER ACCOUNT

يُستخدم Register Flow عندما يريد المستخدم إنشاء حساب جديد داخل النظام. تبدأ العملية بإرسال بيانات التسجيل، وتنتهي بحفظ المستخدم داخل جداول .ASP.NET Identity.

```
Client / Postman -> POST /api/auth/register -> AuthController
-> RegisterDto -> Validation -> Check Existing Email / Username
-> UserManager -> ASP.NET Identity -> Create User -> SQL Database
-> HTTP Response -> Client Receives Message
```

Client / Postman 01

يبدأ المستخدم أو Postman عملية التسجيل ويرسل البيانات المطلوبة لإنشاء الحساب.

مثال على بيانات التسجيل

```
{
  "userName": "khalil",
  "email": "khalil@test.com",
  "password": "123456"
}
```

Register Endpoint POST /api/auth/register 02

هذا هو Endpoint المخصص للتسجيل. معنى الطلب ببساطة: أريد إنشاء حساب جديد.

AuthController 03

استقبال الطلب

يستقبل طلب التسجيل، ويقرأ البيانات المرسلة، ثم يمررها إلى نظام Identity لتنفيذ العملية.

RegisterDto 04

شكل بيانات الإدخال

يستقبل فقط البيانات اللازمة للتسجيل مثل Username و Email و Password بدل استقبال User Entity كامل.

Register: التحقق وإدارة الهوية

VALIDATION - DUPLICATE CHECK - USER MANAGER

Validation 05 التحقق من البيانات

يتأكد النظام أن البيانات المطلوبة صحيحة: الإيميل غير فارغ، وكلمة المرور موجودة، واسم المستخدم موجود، وأن الصيغة مقبولة.

عند وجود مشكلة: يعيد السيرفر غالبًا 400 Bad Request مع أخطاء التحقق.

Check Existing Email / Username 06 منع التكرار

يبحث النظام عن الإيميل أو اسم المستخدم قبل إنشاء الحساب، حتى لا يوجد مستخدمان بنفس بيانات التعريف.

إذا كانت البيانات مستخدمة: تتوقف العملية وتعود رسالة مثل Registration failed.

UserManager 07 إدارة المستخدمين

UserManager هو API أساسي داخل ASP.NET Identity لإدارة المستخدمين.

يمكنه تنفيذ: Create User، Find User، Update User، وإدارة كلمات المرور.

العملية الأهم أثناء التسجيل

```
await _userManager.CreateAsync(user, password);
```

ASP.NET Identity 08 نظام الهوية

نظام جاهز من ASP.NET لإدارة المستخدمين، وكلمات المرور، والRoles، وقواعد الأمان.

مهم جدًا: لا تُخزن كلمة المرور كنص عادي. يقوم Identity بتحويلها إلى PasswordHash آمن قبل تخزينها.

Register: إنشاء المستخدم وإرجاع النتيجة

CREATE USER - SQL DATABASE - HTTP RESPONSE

09 Create User إنشاء الحساب

تتحول بيانات RegisterDto إلى User داخل النظام، وتبدأ عملية إنشاء المستخدم فعليًا.

10 SQL Database حفظ المستخدم

يُحفظ المستخدم داخل قاعدة البيانات، غالبًا ضمن جداول Identity.

أمثلة:AspNetUsers لتخزين المستخدمين، وAspNetRoles لتخزين الأدوار.

11 HTTP Response نتيجة التسجيل

يعيد السيرفر استجابة توضح نجاح العملية أو فشلها.

المعنى	Status
تم إنشاء الحساب بنجاح	200 OK
فشل التحقق أو توجد بيانات مكررة	400 Bad Request

12 Client Receives Message نهاية التسجيل

يستلم العميل النتيجة ويعرض رسالة النجاح أو أخطاء التسجيل.

مثال على Success Response

```
{
  "message": "User registered successfully"
}
```

خلاصة Register: ينشئ مستخدمًا جديدًا، ويستخدم UserManager وASP.NET Identity، ثم يحفظ الحساب داخل قاعدة البيانات.

ثانياً: Login Flow

AUTHENTICATE AN EXISTING USER

يُستخدم Login Flow عندما يكون للمستخدم حساب موجود ويريد الدخول إلى النظام. الهدف هو التحقق من بياناته ثم إصدار JWT Token.

```
Client / Postman -> POST /api/auth/login -> AuthController
-> LoginDto -> UserManager -> Find User By Email -> SignInManager
-> Check Password -> JwtService -> Generate JWT Token
-> HTTP Response -> Client Receives Token
```

بيانات الدخول Client / Postman 01

يرسل المستخدم الإيميل وكلمة المرور إلى الـ API.

```
{
  "email": "khalil@test.com",
  "password": "123456"
}
```

Login Endpoint POST /api/auth/login 02

هذا هو الـ Endpoint المخصص للدخول. معنى الطلب: لدي حساب وأريد تسجيل الدخول.

تسييق عملية الدخول AuthController 03

يستقبل Login Request، ويقرأ الإيميل وكلمة المرور، ويبدأ التحقق منهما، ثم يولد Token عند نجاح العملية.

شكل بيانات الدخول LoginDto 04

يحتوي عادة على الحقول المطلوبة فقط لتسجيل الدخول: Email و Password.

JWT: التحقق وإصدار Login

USER LOOKUP - PASSWORD CHECK - TOKEN GENERATION

05 UserManager البحث عن المستخدم

يُستخدم UserManager أولاً للوصول إلى بيانات المستخدم المسجلة داخل Identity.

06 Find User By Email التحقق من وجود الحساب

```
await _userManager.FindByEmailAsync(email);
```

إذا لم يوجد الإيميل: يعيد النظام 401 Unauthorized لأن المستخدم غير معروف.

07 SignInManager إدارة التحقق

إذا كان المستخدم موجوداً، يستخدم النظام SignInManager للتحقق من محاولة تسجيل الدخول وكلمة المرور.

08 Check Password فحص كلمة المرور

تُقارن كلمة المرور المرسلة مع PasswordHash المخزن باستخدام آليات Identity الأمانة.

```
await _signInManager.CheckPasswordSignInAsync(
    user,
    password,
    lockoutOnFailure: false
);
```

النتيجة: كلمة مرور خاطئة = 401 Unauthorized، صحيحة = متابعة إنشاء Token.

09 JwtService خدمة التوكن

بعد نجاح التحقق، تبني JwtService الـ Claims ومعلومات التوقيع اللازمة لإنشاء JWT Token.

Login: إنشاء التوكن وإرجاعه**GENERATE JWT - HTTP RESPONSE - CLIENT****Generate JWT Token 10** إثبات الهوية الرقمي

يُنشئ النظام Token موقعًا يثبت هوية المستخدم. بدل إرسال الإيميل وكلمة المرور مع كل طلب، يرسل العميل التوكن.

محتوى التوكن: قد يتضمن User ID و Email و Roles ووقت انتهاء الصلاحية، ويكون محميًا بتوقيع رقمي.

HTTP Response 11 إرجاع التوكن

يعيد السيرفر Response يحتوي على JWT Token عند نجاح تسجيل الدخول.

```
{
  "token": "eyJhbGciOiJIUzI1NiIs..."
}
```

Client Receives Token 12 حفظ التوكن

يستلم العميل التوكن ويحفظه لاستخدامه لاحقًا. قد يكون العميل Postman أو Frontend أو Mobile App.

خلاصة Login: لا ينشئ مستخدمًا جديدًا؛ بل يتحقق من مستخدم موجود ويصدر له JWT Token عند نجاح الإيميل وكلمة المرور.

ثالثًا: After Login Flow**USE JWT TO ACCESS PROTECTED ENDPOINTS**

بعد استلام التوكن، يبدأ مسار Authorization عند محاولة الوصول إلى Endpoint محمي.

After Login: الوصول إلى Endpoints المحمية

BEARER TOKEN - AUTHORIZE - PROTECTED API

01 Client Stores JWT Token حفظ التوكن

يحفظ العميل التوكن داخل Postman أو التطبيق حتى يرسله مع الطلبات المحمية.

02 Client Sends Authorization Header إرسال Bearer Token

يرسل العميل التوكن داخل Authorization Header مع كل طلب يحتاج تسجيل دخول.

```
GET /api/car
Authorization: Bearer eyJhbGciOiJIUzI1NiIs...
```

03 Validates Token [Authorize] التحقق من الصلاحية

عند وجود [Authorize] على Controller أو Action، يفحص النظام:

هل التوكن موجود؟ هل التوقيع صحيح؟ هل انتهت صلاحيته؟ وهل Claims وRoles تسمح بالوصول؟

04 Access Protected Endpoint قرار السماح

إذا كان التوكن صحيحًا، يكمل الطلب إلى Controller. وإذا كان غير صالح أو لا توجد صلاحية، يتوقف الطلب.

متى يحدث؟	Status
التوكن مفقود أو غير صحيح أو منتهي	401 Unauthorized
المستخدم معروف، لكنه لا يملك الصلاحية المطلوبة	403 Forbidden

الفكرة: Authentication يثبت من هو المستخدم، بينما Authorization يحدد ما الذي يُسمح له بالوصول إليه.

الفرق والملخص الجاهز للمناقشة

REGISTER VS LOGIN VS JWT

Login	Register
يُتحقق من مستخدم موجود	ينشئ مستخدمًا جديدًا
يستخدم LoginDto	يستخدم RegisterDto
UserManager.FindByEmailAsync	UserManager.CreateAsync
SignInManager.CheckPasswordSignInAsync	Identity + Database
يستخدم JwtService ويرجع JWT Token	يرجع Success Message أو Validation Errors

الفرق المهم جدًا

Register Create a new account	Login Check account and issue JWT	JWT Access protected endpoints
---	---	--

الجملة الذهبية للمناقشة

يبدأ Authentication Flow بتسجيل المستخدم عن طريق Register، حيث يتم إنشاء الحساب باستخدام UserManager و ASP.NET Identity. بعد ذلك يستخدم Login للتحقق من الإيميل وكلمة المرور باستخدام UserManager و SignInManager، وإذا كانت البيانات صحيحة يتم إنشاء JWT Token. بعدها يرسل العميل هذا التوكن مع الطلبات المحمية، و [Authorize] يتحقق من التوكن قبل السماح بالدخول.

أبسط نسخة للحفظ

REGISTER
Create User

LOGIN
Check Email + Password
Generate JWT

AFTER LOGIN
Send Bearer Token
[Authorize]
Access API

الخلاصة: Register ينشئ الهوية، Login يتحقق منها ويصدر Token، و JWT يسمح للعميل بإثبات هويته عند طلب الموارد المحمية.